

DinkDonk

An RL-trained Agent Model for Pickleball Game in Unity

Jack and Zaraius

Abstract

Most topics in Machine Learning and Artificial Intelligence are complicated and unfriendly to beginners once one starts to investigate the math behind each concept. This disconnect between simple intuition and deep understanding often leads to limited freedom of applications and a higher barrier to modifying fundamental concepts to more complex scenarios.

However, as we will explore in this report, Reinforcement Learning stands out as a more straightforward, intuitive, and engaging topic that provides many interesting applications to real-world contexts. This is especially reflected in a gamified or simulated context, where many real-world problems can be simulated virtually and then explored more systematically. Examples include training a robot how to walk like a human by performing RL with well-designed rules, rewards, and penalties. Without the need to understand all the logic behind, one can not only immediately see visual results, but also get a variety of unexpected behaviors of the model performing some tasks that a human might do easily.

Therefore, we will be exploring the approach and process to develop an RL-trained model to play pickleball in the Unity game engine. We chose this as our topic of research, both because it is a unique topic that has not been done by others before, and it allows us to learn designing a Unity game environment. Pickleball serves as a good physics environment from real life, where it is a multiplayer sport that involves multi-body physics and many degrees of freedom, which is complex enough for our purpose of exploring how well the RL-trained agent learns to play this sport. From this project, we hope to gain insights into the development process of training an RL model, game design in Unity, and general Machine Learning skills in iterating on a model.

Setup

The training and setup environment are all done in the game engine Unity. As shown in Figure 1, elements in the environment include the player holding a paddle, a pickle ball, a net, the ground, and finally the walls.

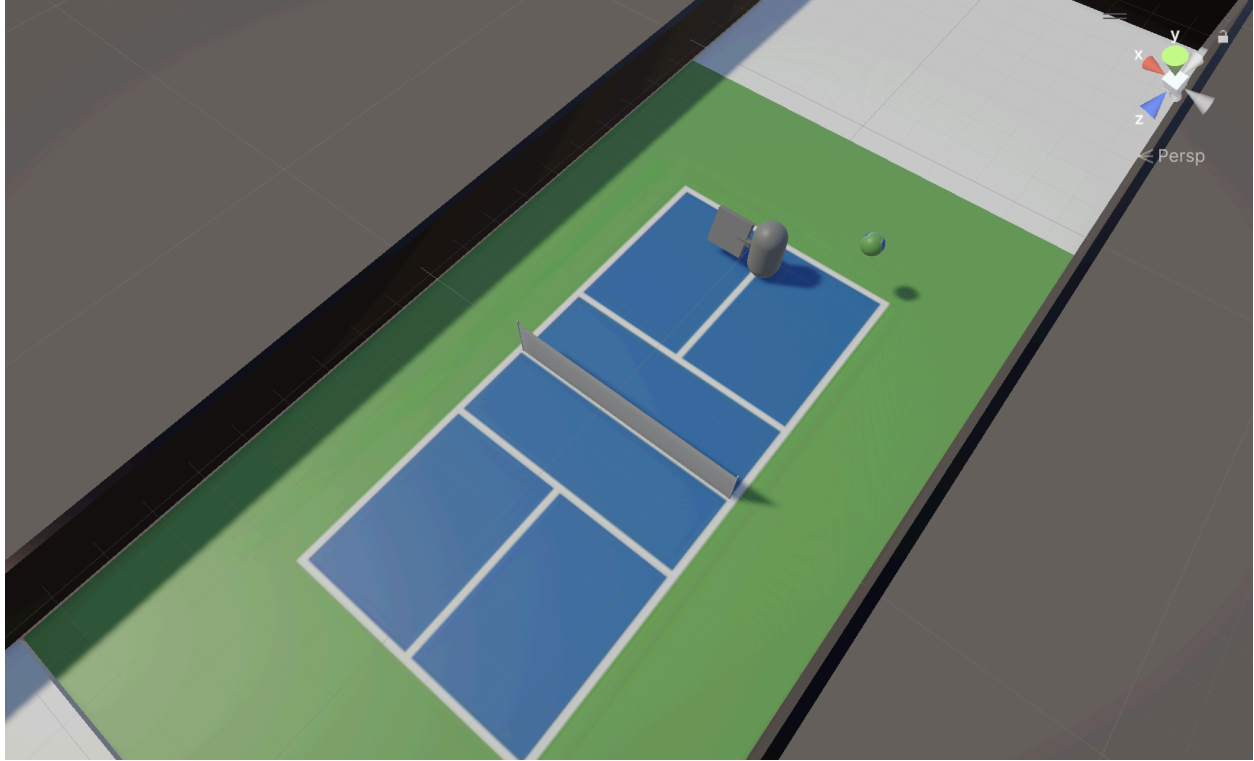


Figure 1: An image overview of the Unity environment setup.

Given the scope and time for this project, it is difficult to simulate the exact physics environment involved in the sports of pickleball, including the multi-joint system of a human holding a paddle, hitting a ball. Therefore, to serve the main learning goal of this project, we have decided to simplify the physics to an appropriate level so that the core interactions are kept, and RL training can be applied. For example, instead of an exact human body, the player agent is represented in a simplified cylindrical body that is just capable of 2D translation. As shown in Figure 2, to simplify the complicated multi-joint system of an arm holding a paddle, we design the paddle to be fixed at the right side of the player, floating in the air, and only have one degree of freedom for rotating around the x-axis, which is tilting. The net is also simplified to be only a rigid body instead of a fluid, extensible net like in real life. These simplifications allow us to explore the results of RL training in this environment without the loss of generality.

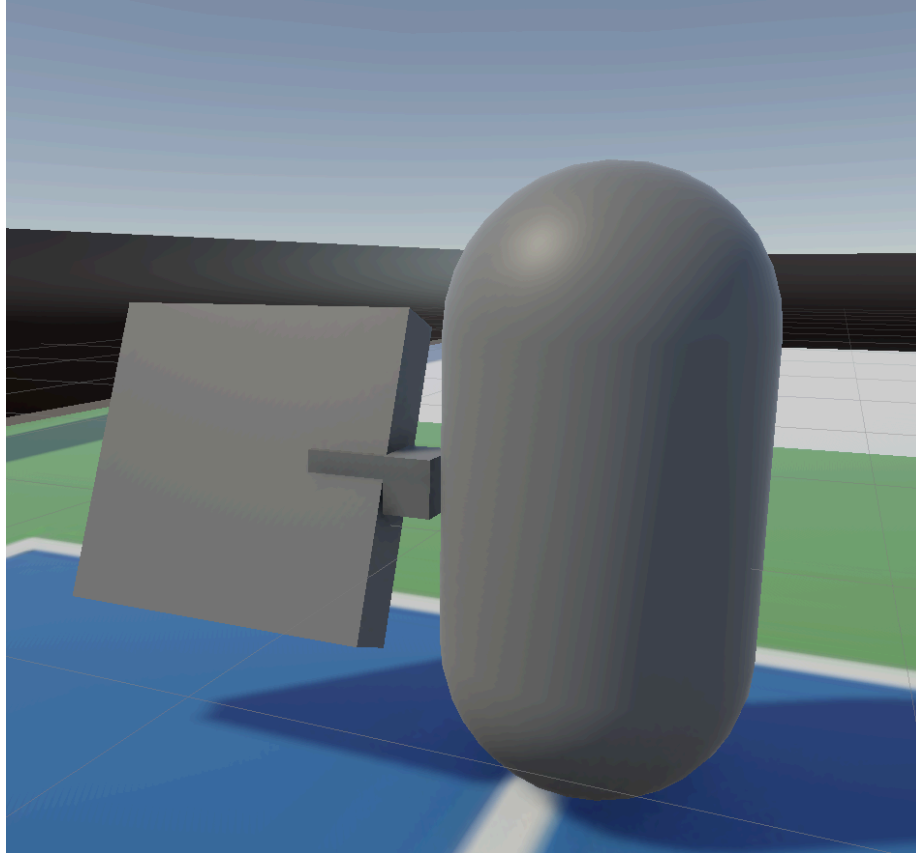


Figure 2: A close-up illustration of the player agent with the paddle fixated at the right side of the player.

In terms of the RL training, we use a convenient library called [ml-agents](#) for all the training and hyperparameter-tuning, which integrates seamlessly with the Unity engine. It allows us to configure the reward-penalty rules directly inside the C# scripts for each game object, as well as the configurations for the training. Many statistical graphs and visuals are also generated automatically regarding the training performances, which can be accessed by executing the command `tensorboard --logdir=results`.

For documentation purposes, as well as consistency and reproducibility across machines, we use Python 3.10.12 with Conda, and `mlagents==1.1.0` for this report. These are the compatible versions for the latest ml-agents release, and they may change in the future as new releases come out. More details on setting up ml-agents for a Unity environment can be found [here](#).

Methods

To reach our ultimate goal of training an agent that mimics a real human player playing pickleball, we have two approaches: We can either train one agent alone by manually serving the ball towards it, or take a more adversarial approach where we have two agents learn by playing against each other.

We will cover the development process of both approaches and have a comparison of their performances separately at the end.

Single Agent Training

For the single-agent training, since there is no other player, we need to manually serve the ball to the agent to start. We draw inspiration from the real pickleball rule and always serve the ball from the back line of one of the boxes on the opponent's side to the diagonal box of the agent's side, as shown in Figure 3.

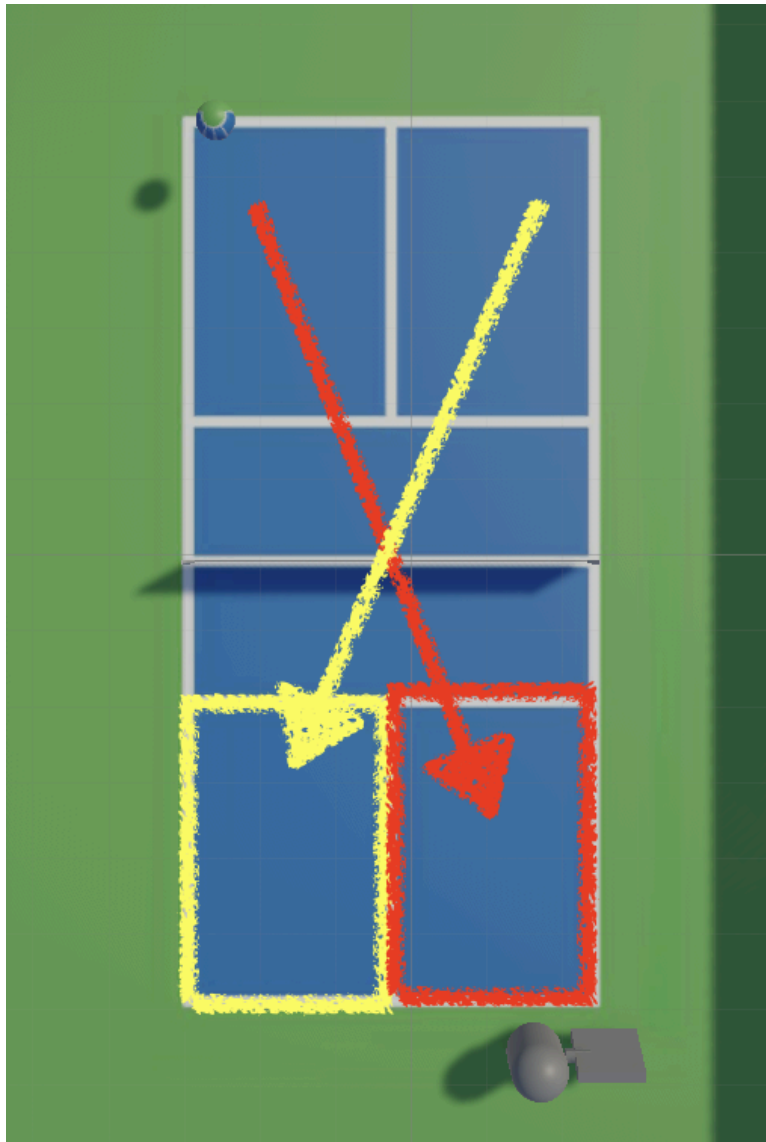


Figure 3: A top-down view demonstrating the serving setup for single-agent training

The ball is served by simply applying a force to the ball. However, to randomize the serving situation as much as possible for the agents to learn how to return a ball, we calculate this serving force in a reverse-engineering way: We first randomly choose a starting point for the ball to spawn at on the opponent's side back line. Then, based on which box the ball is currently in on the opponent's side, we select the diagonal box on the player's side, and randomly generate a landing point for the ball in the selected box. Given the 3D coordinates of the starting and finishing point of the ball's trajectory, we reverse calculate the force necessary to serve the ball to ensure that it lands in bounds every serve.

Once the ball is served, the agent will learn to return the ball by trying out different inputs into the agent's motion and improve based on the total rewards at the end of each episode. The inputs to the agents include 2D translation on the xz-plane and the tilting angle of the paddle. At every timestamp, the agent will attempt numerous combinations of input and explore which way to maximize the rewards. But due to the level of freedom in this environment, the agent can easily just move around a single point without making much exploration or progress. To avoid this, we first set a maximum step number for each episode to 1000, which prevents a session from taking too long by immediately ending the episode once the agent lingers too long without getting any rewards. We also set up walls around the court that encourage the agent to stay within the court instead of running out of the court. Whenever the agent hits the wall, it will be penalized and end the episode immediately.

Now, we set up the reward-penalty systems around the specific rules about pickleball for the agent to learn how to play properly. As shown in Figure 4, the table shows all of the reward-penalty systems we finalized at the end. Many of the rewards are directly inherited from the game rules: Penalties are applied if the return lands out of bounds, hits the ball more than once, or the ball bounces twice on the court, etc. On the other hand, rewards will be given if the player's paddle hits the ball and the return lands in bounds on the opponent's side.

We now have enough details to start the training process for the agents. The configuration details for the training can be found in [results/config](#). In terms of key hyperparameters, the training uses a total of 3 hidden layers, 256 units each. The batch size is 2048, with a learning rate of 0.001. The configurations are kept relatively consistent throughout the development process because they are shown to be less significant compared to the reward-penalty systems. We also create multiple copies of the same environment game fields in Unity to speed up the training process.

There were a few obstacles and updates made during the development of the training environment. First of all, we observed that in the very first version, even with a clear rule of the game, the agent did not learn how to hit the ball at all. The agent randomly explored the game field and the training took too long to produce any observable success of hitting the ball. Therefore, to incentivise the agent to hit the ball with a much better chance, we added a dynamic reward, where it would be rewarded 0.05 points if the agent gets closer to the ball every timestamp. This reward was intentionally set to be small so the agent does not endlessly chase after the ball all the time while providing guiding directions for where to go to hit the ball.

However, we still observed the agent sometimes going through the rigid net to continue chasing after the ball after returning, as shown in Figure 4. We then added a penalty for the player to cross the net to prevent it from happening. These adjustments turned out to significantly help the performance of the agents approaching the ball and getting a hit on the paddle first.

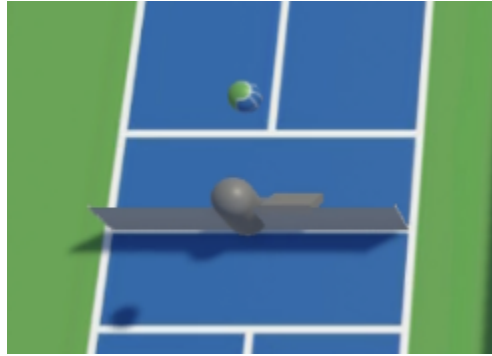


Figure 4: An image showing the agent crossing through the rigid net illegally to chase after the ball as a result of the small reward for approaching the ball

The agent can now learn how to hit a ball at a much faster rate. Within one minute of training, we can start seeing the agents consistently attempting to hit the ball.

After a longer period of training, the agent learned to not only hit the ball with the paddle, but also returned to the opponent's court properly. However, after further training, we observed that the agent tried to maximize the rewards in each round instead of following the game guideline. As shown in Figure 5, since rewards will be given whenever the ball collides with the paddle, the agent learned to catch the serve by balancing the ball on the top of the paddles so collisions were constantly detected, leading to unreasonably high rewards. To prevent this, we modified the training so that the agent would be penalized if the ball stays on the paddle for too long. Not surprisingly, the agent stopped performing the trick of ball-balancing on paddles.



Figure 5: An image showing the agent holding the ball on the paddle still to maximize the rewards instead of returning the serve

Up to this point, the agent was able to achieve a very high return success rate. But most of the returns were very high, as shown in Figure 6, which was monotonous and would not pose a threat in a real pickleball game. Therefore, similar to the rewards for incentivizing approaching the ball, we also added a small reward to encourage the agent to return the ball with a lower Y

value. The lower the Y value above the net, the greater the rewards. This change improved the return height slightly. Many returns had a lower height and higher quality.

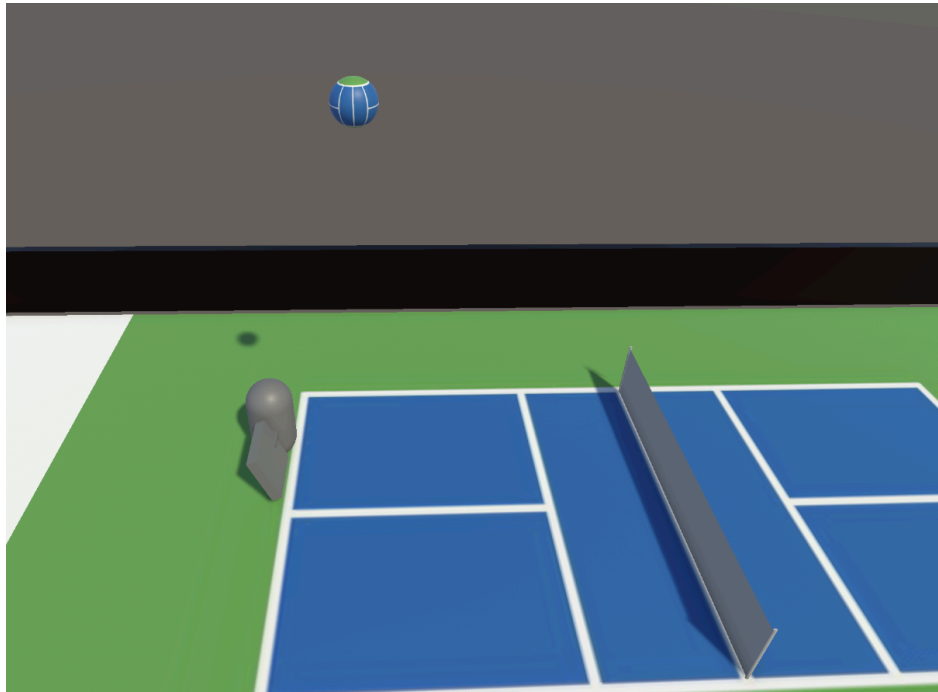


Figure 6: An image showing the behavior of the agent tending to return the ball high in the air on average

Table 1 summarizes all of the final reward-penalty rules used to get our final agent model for replication and documentation purposes.

Event	Reward
Ball lands on opponent's side (first bounce, in bounds)	+100
Hit the ball	+20
Moving closer to ball (each frame)	+0.05
Ball stuck on ground (10+ frames)	-5
Ball held on paddle too long	-20
Hit ball twice on your own side	-20
Hit wall trigger	-50
Player crossed the net	-50
First bounce out of court bounds	-150
Double bounce — failed to return	-150
You hit ball out (fell below $y < -5$)	-150

Table 1: Reward-penalty summary table for the final agent model training

Multi-Agent Training

Our goal for multi-agent training was to move beyond a single agent returning a manually served ball and instead create a setting where two agents could learn to rally with each other. In this setup, both players are placed on opposite sides of the court, and each episode begins with the ball being served from one side toward the other. The serve is still generated through the same reverse-calculated trajectory method described earlier, but it is randomized so either side can begin the rally. This prevents the model from only learning how to respond from one fixed court position.

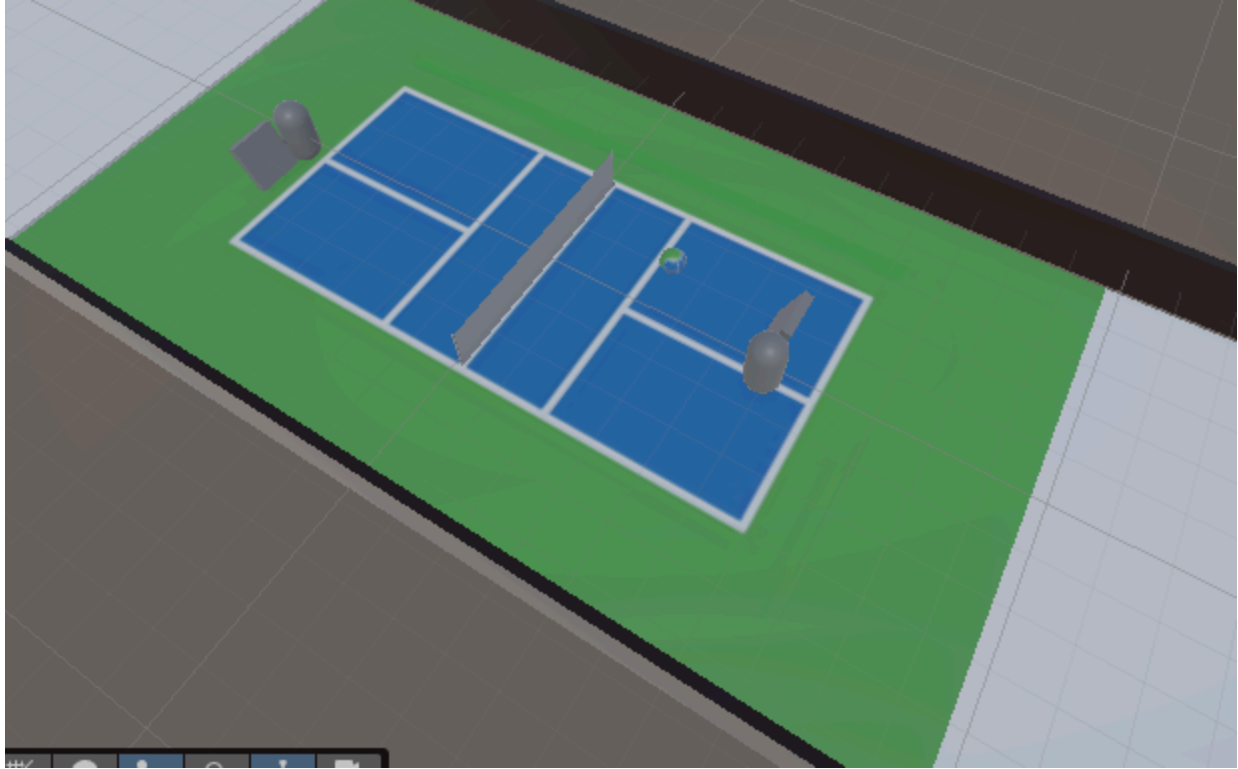


Figure 7: An Image showing the Unity setup for 1v1

For this part of the project, we explored self-play as the main training idea. In self-play, the agent does not learn only against a fixed opponent. Instead, it learns by playing against versions of itself. As training progresses, older versions of the model are saved and reused as opponents, while the current model continues to improve. This creates a moving but still learnable target: the agent is challenged by opponents that are similar enough to be playable, but different enough to expose weaknesses in its current behavior. In ML-Agents, this is supported through the self-play configuration, where saved policies can be swapped in as opponents during training.

A major change needed for multi-agent training was making the observations work correctly for both sides of the court. In the single-agent version, the agent only needed to understand the ball and its own position from one side. With two agents, the same policy may control players on opposite sides, so the observations must be symmetrical. To handle this, we converted the agent position, ball position, and ball velocity into a team-relative coordinate system. This means that each agent observes the world as if it is playing from its own consistent perspective, rather than requiring the neural network to separately learn two mirrored versions of the same task.

We experimented with increasing the amount of information given to the agents during development. Earlier versions included more detailed observations, such as opponent-related information and additional state variables. This was useful for debugging because it made the environment more explicit. However, it also made the learning problem more complicated and did not always improve the quality of the final behavior. After testing, we reduced the observation space back down to the most important information: the agent's position, the ball's position, and the ball's velocity. This final observation design kept the model closer to the simpler single-agent setup while still giving it enough information to move toward the ball and respond to incoming shots.

The reward system was also adjusted for rallies instead of isolated returns. The agent still receives rewards for hitting the ball and for landing the ball successfully on the opponent's side, while receiving penalties for illegal or failed behavior such as hitting the wall, crossing the net, double-bouncing, sending the ball out of bounds, or holding the ball on the paddle. In addition, we added a rally continuation reward that increases as the number of successful hits in the rally grows. This encouraged the agents not only to win a point immediately but also to sustain longer exchanges across the net.

Event / Behavior	Reward	Ends Rally?	Purpose
Paddle hits ball	+20	No	Encourages the agent to make contact with the ball
Rally continuation bonus	+3 × rally hit count, capped at 10 hits	No	Encourages longer rallies in the trained shared-policy model
Ball reaches opponent half after hit	+5	No	Rewards sending the ball across the net
Ball lands legally on opponent side	+100	No	Rewards successful, in-bounds shots
Ball falls below court after agent hit	-150	Yes	Penalizes failed returns
First bounce out of bounds	-150	Yes	Penalizes illegal shots
Double bounce	-150	Yes	Penalizes failing to return the ball
Agent hits wall	-50	Yes	Penalizes leaving the playable area
Agent crosses net	-50	Yes	Keeps each player on their own side
Ball held on paddle too long	-20	Yes	Prevents exploiting paddle contact
Double hit on own side	-20	Yes	Penalizes illegal repeated hits
Ball stuck on ground	-5	Yes	Ends unproductive stuck states
Paddle aligned with playable incoming ball	+0.02	No	Helps agent position for returns
Improved lateral alignment with ball	-0.01 to +0.02	No	Rewards moving closer to the predicted ball path
Moving horizontally toward incoming ball	+0.005	No	Encourages useful movement
Moving horizontally away from incoming ball	-0.005	No	Discourages bad positioning

Figure 8: Summary of Rewards and Penalties for 1v1 Agent Training

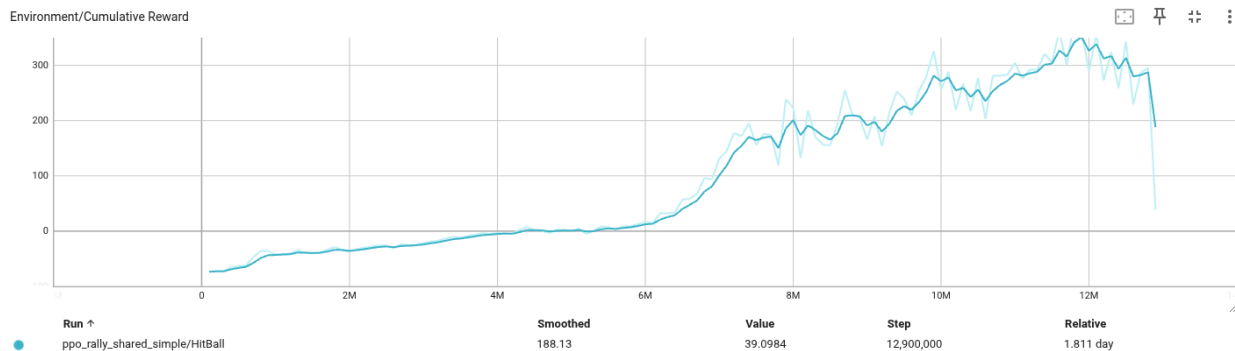


Figure 9: Graph of Cumulative Reward through training

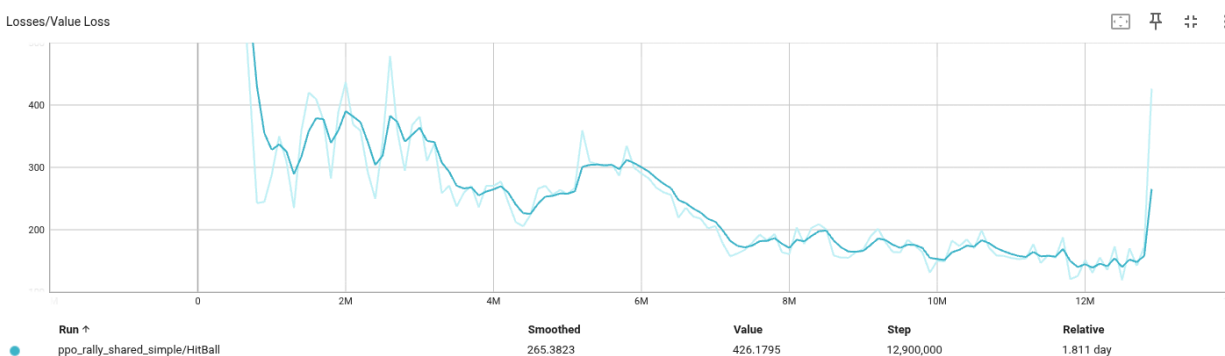


Figure 10: Graph of Loss through training

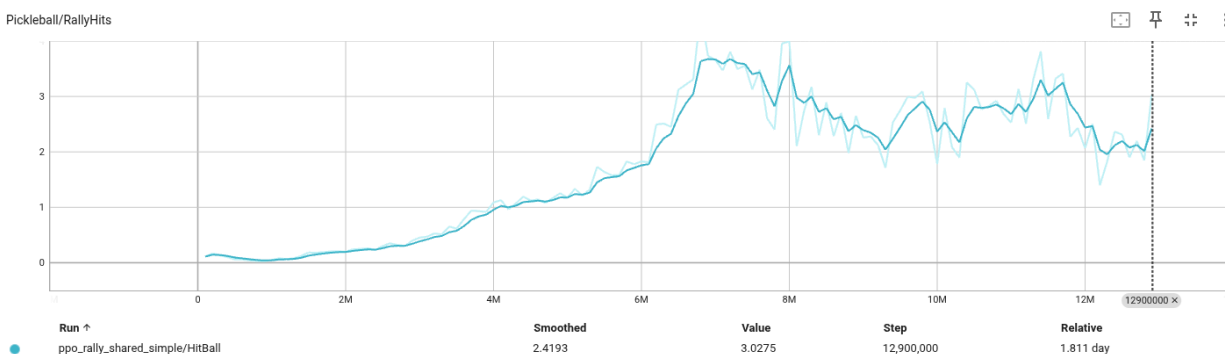


Figure 11: Graph of Average Rally Count through training

One difficulty we observed was that the fully self-play version was less visually stable than expected. Because both agents were learning at the same time, the environment became more complex than the single-agent case. If one model learned an unusual behavior, the other agent would adapt to that behavior, sometimes producing strategies that technically increased reward but did not look like realistic pickleball. To address this, we also trained a simplified shared-policy rally version. In this version, both agents used the same policy and learned under symmetrical conditions. This produced the best visual rally behavior overall, especially in the

checkpoint around 11,999,982 steps, which showed more convincing movement and returns between the two players.

Overall, the multi-agent training demonstrated that rally behavior is possible in this simplified pickleball environment, but it is significantly harder than single-agent return training. The single-agent model can focus on one clear task: reach the ball and return it in bounds. The multi-agent model has to learn a longer interaction pattern where both players affect the future state of the rally. This made the training more sensitive to observation design, reward shaping, and the balance between competitive and cooperative incentives.

Results

Both approaches have different forms of end results that can be hard to compare with each other. But we can look at the success rate of returns from both agents. For the single agent, it can properly return 84% of the serves. This is reasonably high given that some of the serves have a fast speed and extreme angles that make them hard to return in the first place.

For the multi-agent training, we benchmarked the total number of hits within an episode, where we got an average of 4, but a max of 40. This huge standard deviation is because the model loses often by 0 or 1 hit due to things like double bounce or crossing the net. However, if it is able to make it past this point, it does very well and can reach long rallies.

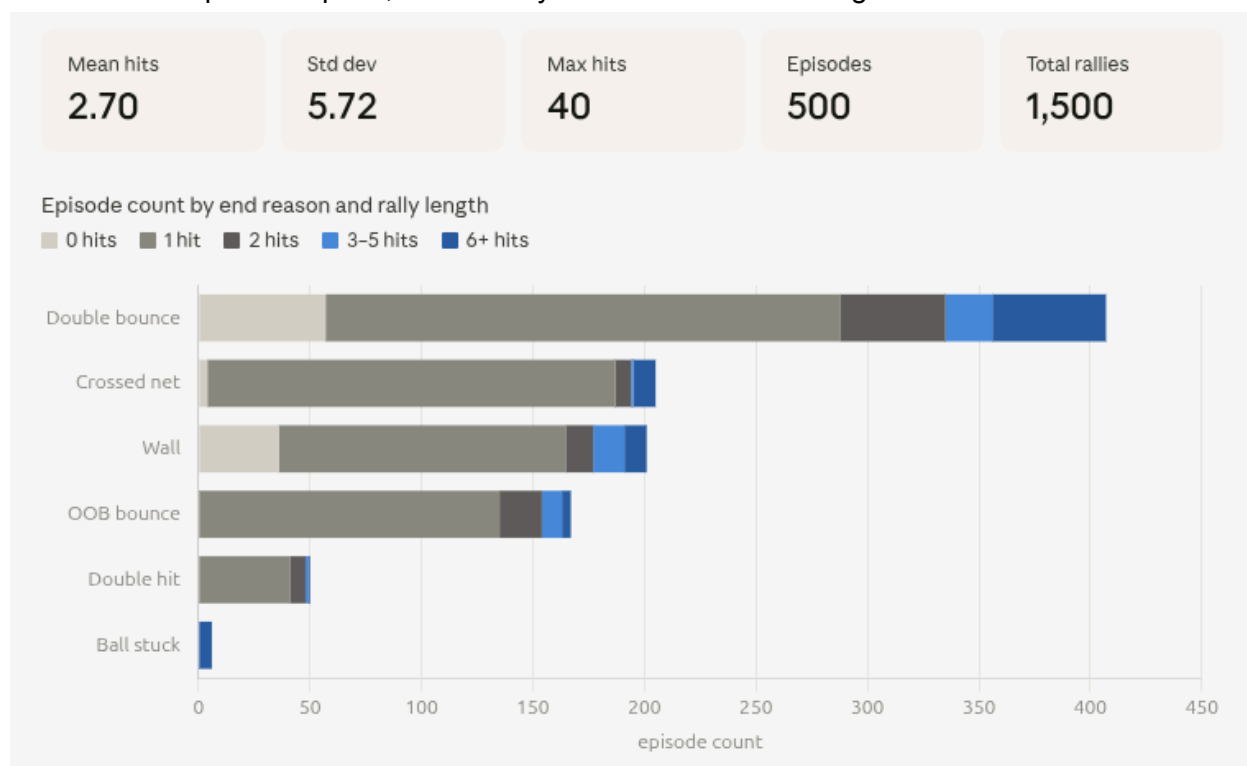


Figure 12: Graph showing common deaths and when it occurs

Future Discussions

There are several natural directions for improving this project in future work. The first is to make the physics environment closer to real pickleball. Our current environment simplifies the player as a cylindrical body with a fixed paddle position, which makes training much more manageable. However, a more realistic version could include arm movement, paddle swing timing, player acceleration limits, and more accurate paddle-ball collision physics. These additions would make the learned behavior more realistic, though they would also make the training problem much harder.

Another important improvement would be refining the reward system for multi-agent rallies. The current rewards successfully encourage the agents to hit the ball and keep rallies going, but they do not fully capture shot quality. In real pickleball, a good return is not only in bounds; it may be low over the net, placed away from the opponent, or timed to make the next return difficult. Future versions could include rewards for strategic placement, lower shot height, controlled velocity, and forcing the opponent into a difficult position. This would help move the agents from simply rallying to actually playing competitively.

The self-play setup could also be expanded. In this project, self-play was useful but unstable because both sides were learning in a changing environment. A more advanced training process could use a larger pool of past models, more careful opponent sampling, or curriculum learning. For example, the agent could first learn against an easy scripted opponent, then against older saved models, and finally against the latest version of itself. This gradual increase in difficulty may produce more stable learning and more natural gameplay.

Finally, future work could include a more detailed quantitative evaluation of the trained models. In addition to TensorBoard graphs such as cumulative reward, episode length, ELO, and rally hits, we could measure return success rate, average rally length, out-of-bounds frequency, net-crossing frequency, and shot height. These metrics would make it easier to compare the single-agent model, the self-play model, and the shared-policy rally model more objectively. Based on our observations, the shared-policy multi-agent model produced the best visual rally behavior, while the single-agent model was more reliable at the narrower task of returning serves.

Appendix

Github Link: <https://github.com/Zaraius/DinkDonk> (main branch has serve return, 1v1WithML has 1v1 mechanic)